

Architettura degli Elaboratori

Le istruzioni della CPU – parte 1



SAPIENZA
UNIVERSITÀ DI ROMA

[S&PdC] 2.1 – 2.5



Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Claudio Di Ciccio,
Iacopo Masi, e Andrea Sterbini

Logistica e Comunicazioni

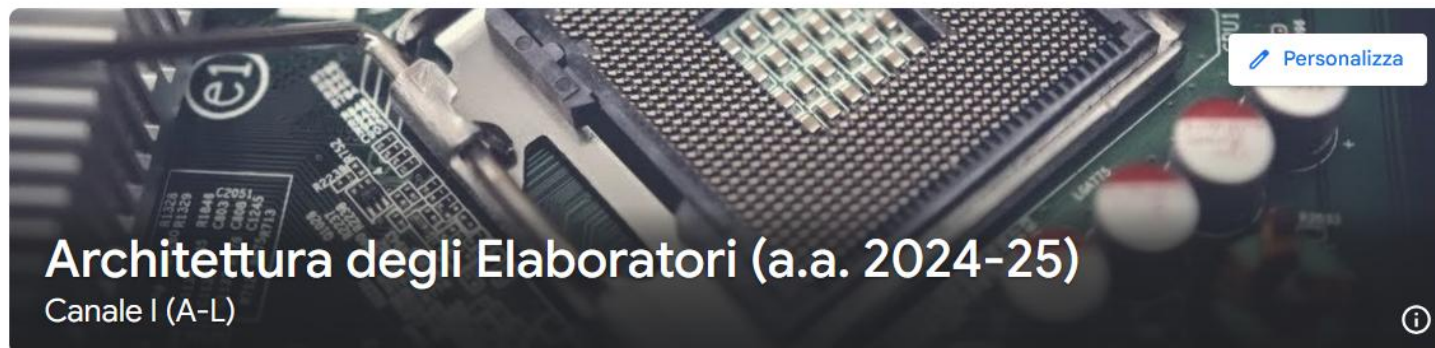
Accesso al materiale e comunicazioni: Google Classroom

Stream

Lavori del corso

Persone

Voti



Meet

Genera link

Codice del corso

3gyvx3v

In consegna

Nessun lavoro in consegna
a breve

Visualizza tutto



Pubblica un annuncio per il tuo corso



Qui inserirai le comunicazioni per il tuo corso

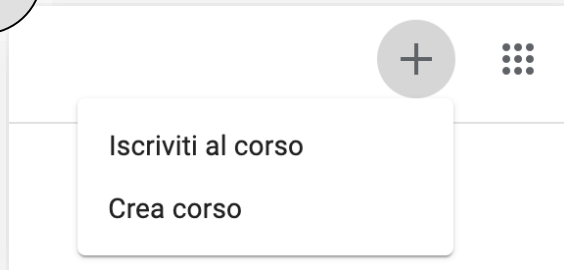
Usa lo stream per condividere annunci, pubblicare compiti e rispondere alle domande degli studenti

Impostazioni dello stream

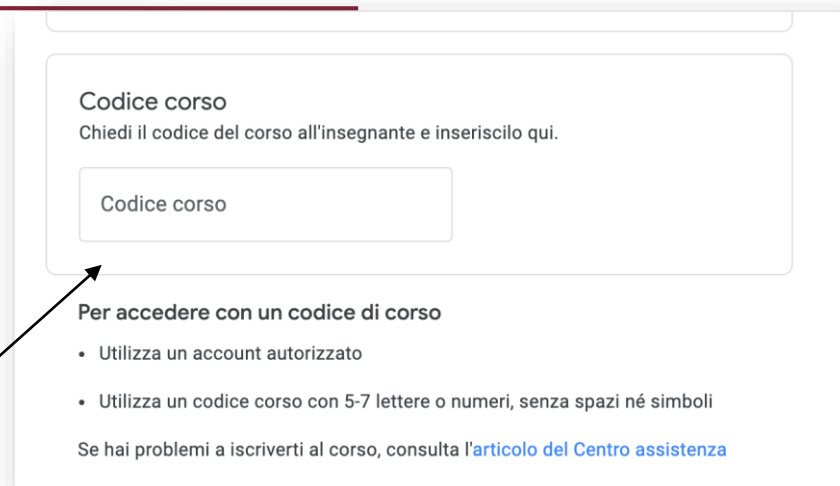


Google Classroom

1



codice



2

Il codice (o link di registrazione) è anche in bacheca:

3gyvx3v

Calendario delle lezioni

Controllare il calendario per potenziali cambi di programma. Se siete iscritti a Google Classroom del corso riceverete una notifica.

	DOM	LUN	MAR	MER	GIO	VEN	SAB
	2	3	4	5	6	7	8
GMT+01				Mercoledì delle Ceneri			
7 AM							
8 AM							
9 AM							
10 AM							
11 AM			[Teaching] [AE] Architettura degli Elaboratori 11AM – 2PM				
12 PM						[Teaching] [AE] Architettura degli Elaboratori 12 – 2PM	
1 PM							
2 PM							
3 PM							

- Lezione sempre in questa aula
- Pausa ogni ora circa

Argomenti

Motivazioni

Nel resto del corso vedremo nel dettaglio la progettazione di una CPU RISC-V, che:

- Ha un set di istruzioni ristretto (**RISC**)
- È volutamente semplice
- È facile da velocizzare con la pipeline
- È facile da parallelizzare
- È un esempio eclatante di progettazione coordinata dell'hardware col software

Esempi di CPU con architettura RISC:

- **SPARC** di Sun, **RISC-V** in sistemi embedded
- **PowerPC** nei Mac e server IBM
- **Cell** delle Play Station Sony
- **ARM** di quasi tutti i cellulari e tablet

Argomenti della lezione

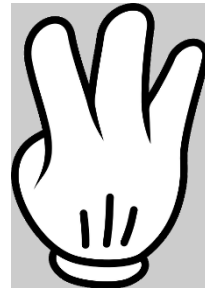
- Rappresentazione dei numeri
- Interi con Segno (complemento a 2)
- Floating point / Double
- Formati delle istruzioni RISC
- Primi cenni di istruzioni RISC-V

Ripasso, Preliminari, Sistema Posizionale



SAPIENZA
UNIVERSITÀ DI ROMA

Contare



Bits, nibbles e bytes

Un **binary digit**, o **bit**, è un elemento di un insieme binario di simboli.

Usiamo:

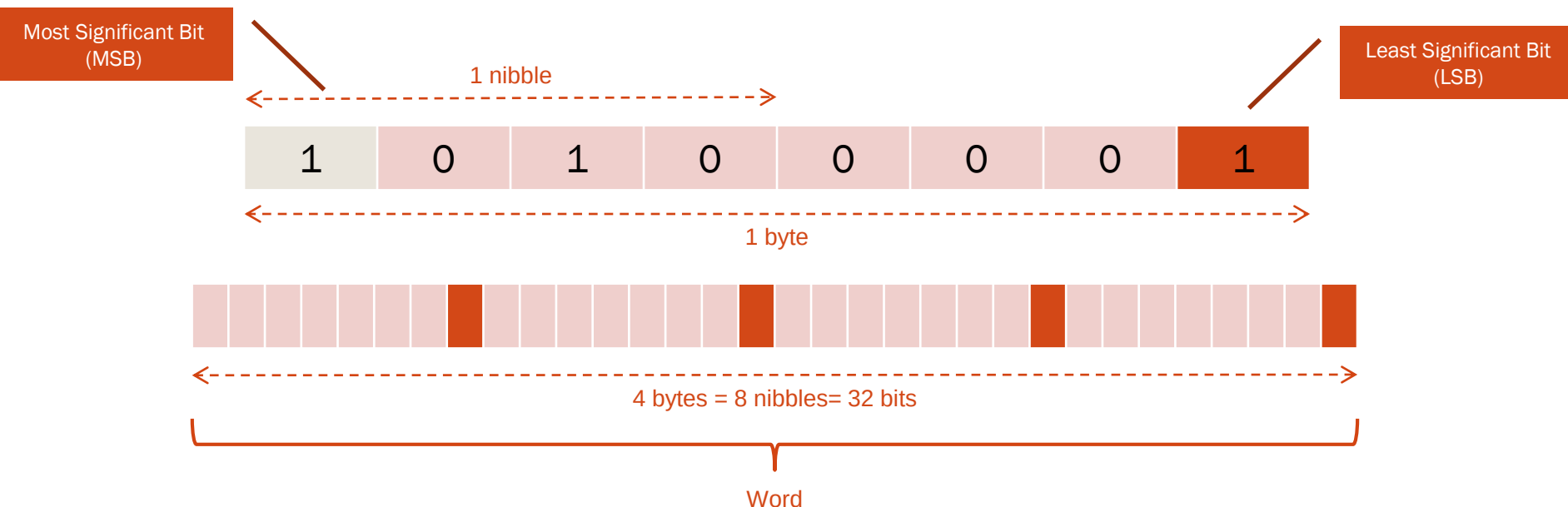
0 (zero), **spento, falso**

1 (uno), **acceso, vero**

Un **nibble** è una sequenza di 4 bit

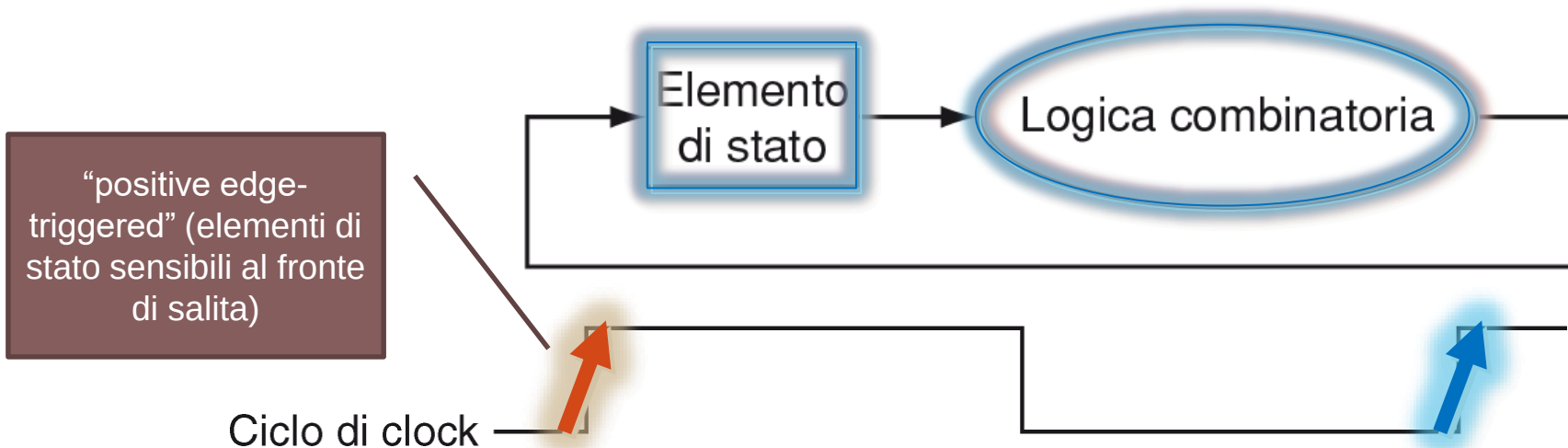
Un **byte** è una sequenza di 8 bit

In architetture a 32 bit, 32 bit $\rightarrow$ 4 byte $\rightarrow$ 1 **word**



Perché solo cifre binarie?

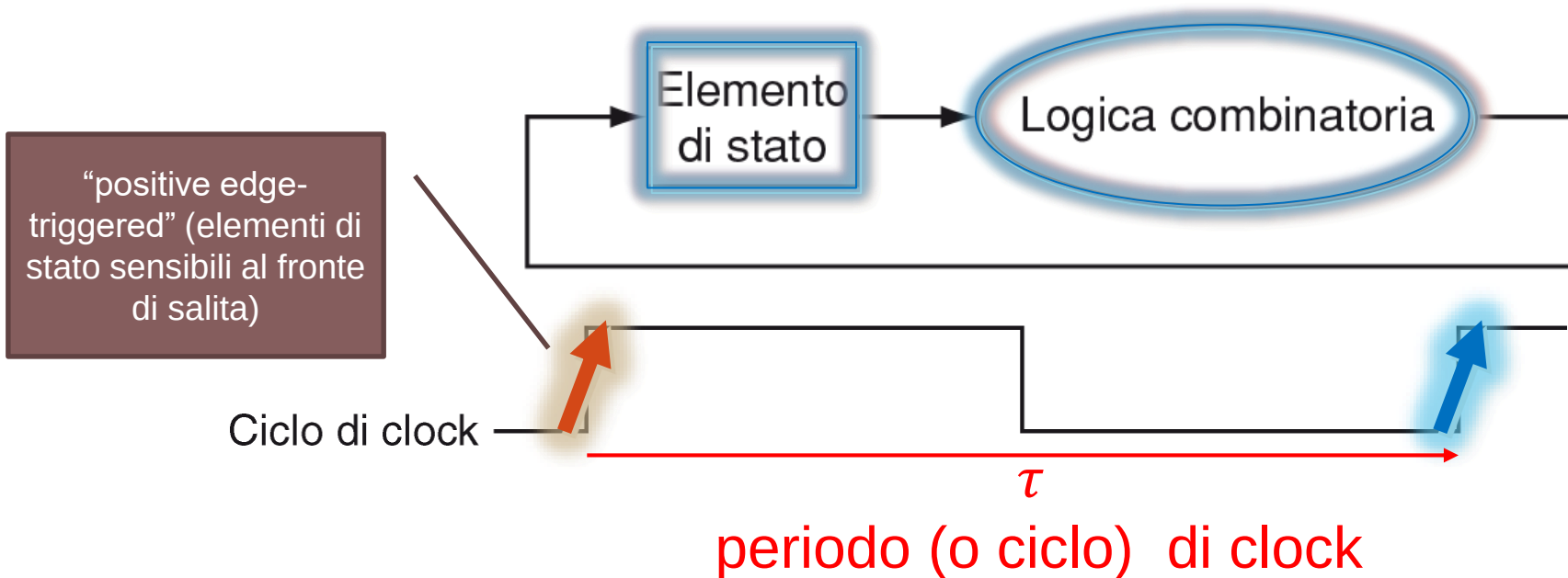
CPU = macchina sequenziale ovvero Stato + circuito combinatorio



Nel sistema binario ogni numero può essere scritto con due soli segni, lo 0 e l'1, facilmente traducibili, come dicevamo, in **segnali elettrici per il calcolatore** e che questo esegue rapidamente operazioni anche molto complesse.

Perché solo cifre binarie?

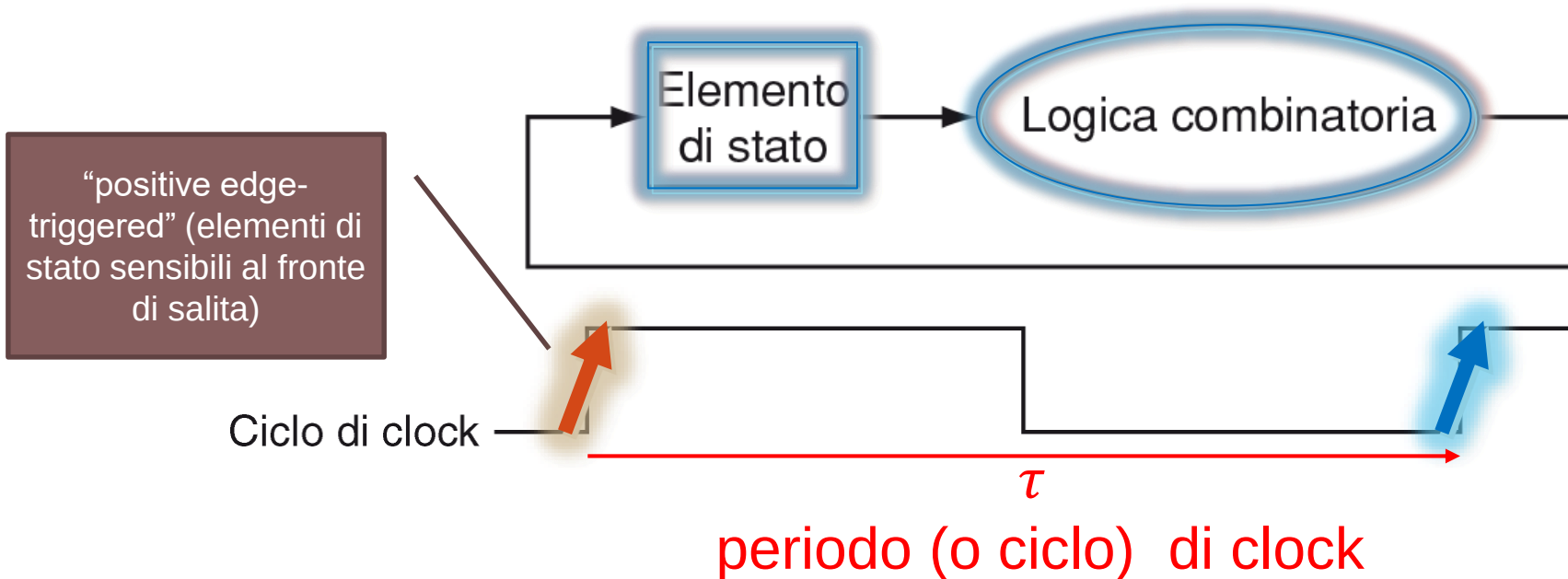
CPU = macchina sequenziale ovvero Stato + circuito combinatorio



Nel sistema binario ogni numero può essere scritto con due soli segni, lo 0 e l'1, facilmente traducibili, come dicevamo, in **segnali elettrici per il calcolatore** e che questo esegue rapidamente operazioni anche molto complesse.

Perché solo cifre binarie?

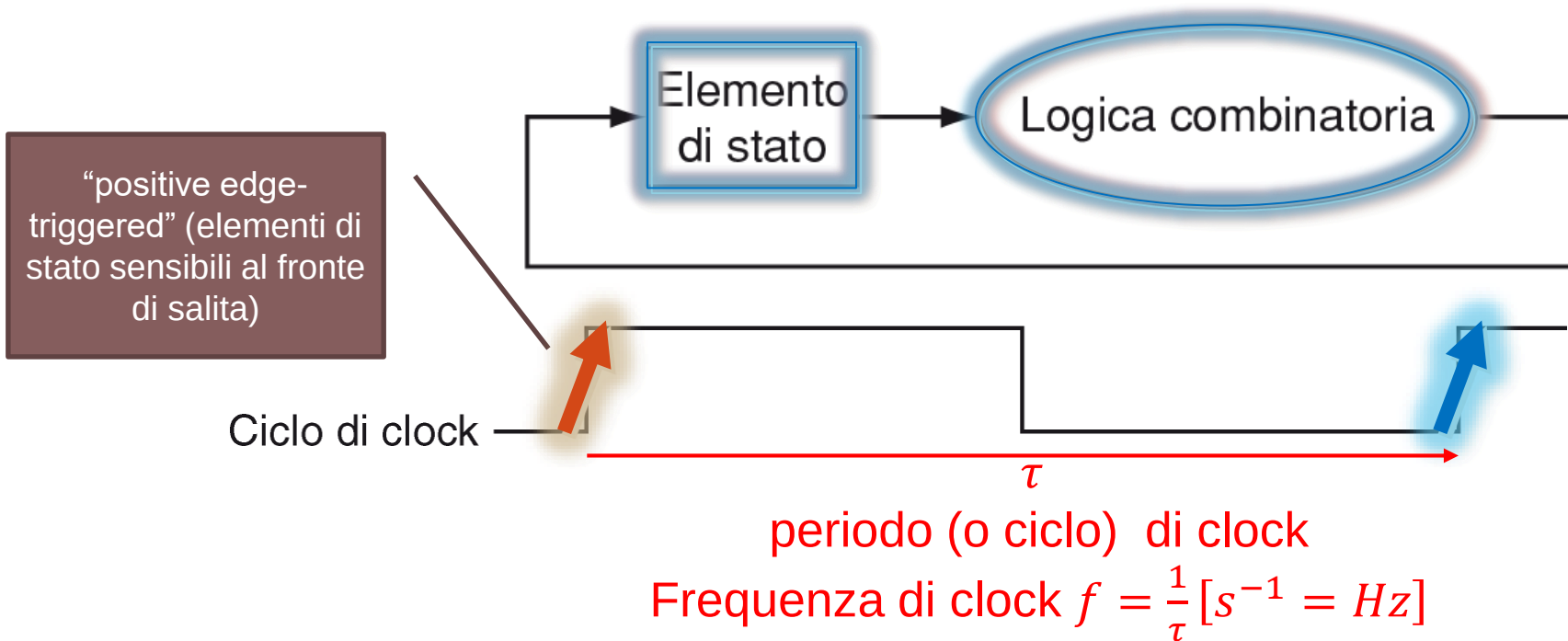
CPU = macchina sequenziale ovvero Stato + circuito combinatorio



Ricordate che le prestazioni **NON** dipendono solo dalla velocità del clock.
Il calcolatore è un sistema complesso.

Perché solo cifre binarie?

CPU = macchina sequenziale ovvero Stato + circuito combinatorio



Ricordate che le prestazioni **NON** dipendono **solo** dalla velocità del clock.
Il calcolatore è un sistema complesso.

Ripasso: In generale il tempo di esecuzione di un programma dipende da

1) # di istruzioni di un programma 2) CPI (#medio di clock per istruzione) 3) velocità clock

Notazione posizionale

Formula generale per il calcolo del valore W :

B : base

n : numero di cifre (digits)

b_i : valore della cifra i -esima (valore nominale)

B^i : position weight

$$W = \sum_{i=0}^{n-1} b_i \times B^i$$

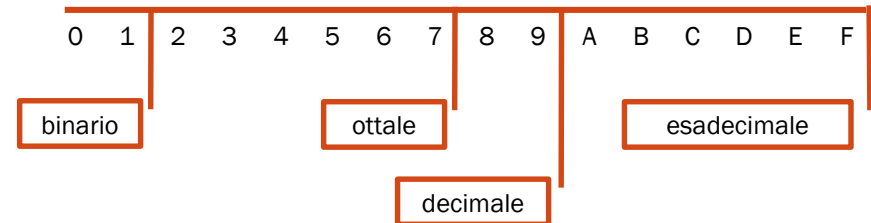
Esempi di notazione:

Sistema decimale (base 10) -> 161

Sistema binario (base 2) -> 10100001_{due}

 Sistema ottale (base 8) -> 241_{otto}

Sistema esadecimale (base 16) -> A1_{esa}



Spesso gli esadecimali sono identificati da prefisso 0x

e.g., 0xA1

Conversione tra sistemi

Da binario a decimale:

$$1100_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

Conversione tra sistemi

Da binario a decimale:

$$\overset{3}{1}\overset{2}{1}\overset{1}{0}\overset{0}{}_2 = 1 \times 2^{\textcircled{3}} + 1 \times 2^{\textcircled{2}} + 0 \times 2^{\textcircled{1}} + 0 \times 2^{\textcircled{0}} = 8 + 4 + 0 + 0 = 12$$

Conversione tra sistemi

Da binario a decimale:

$$1100_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

Da decimale ad esadecimale:

161	:	16	=	10	q	Resto:	1
10	:	16	=	0		Resto:	A

Result: 0xA1

divisione intera

$$x // y \rightarrow x = y \cdot \underbrace{q}_{\text{quoziente}} + \underbrace{r}_{\text{resto}}$$

Conversione tra sistemi

Da binario a decimale:

$$1100_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

Da decimale ad esadecimale:

digit position
// ← 1)

161 : 16

= 10q

Resto: 1r

10 : 16

= 0

Resto: A

Result: 0xA1₀

divisione intera

$$x // y \rightarrow x = y \cdot \underbrace{q}_{\text{quoziente}} + \underbrace{r}_{\text{resto}}$$

Conversione tra sistemi

Da binario a decimale:

$$1100_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

$$1100_2 \rightarrow 12_{10}$$

Da decimale ad esadecimale:

digit position
// ← 1)

$$\begin{array}{rcl} 161 & : & 16 \\ 10 & : & 16 \end{array} = \begin{array}{rcl} 10 & & \\ 0 & & \end{array}$$

divisione intera

$$x // y \rightarrow x = y \cdot q + r$$

quoziente resto

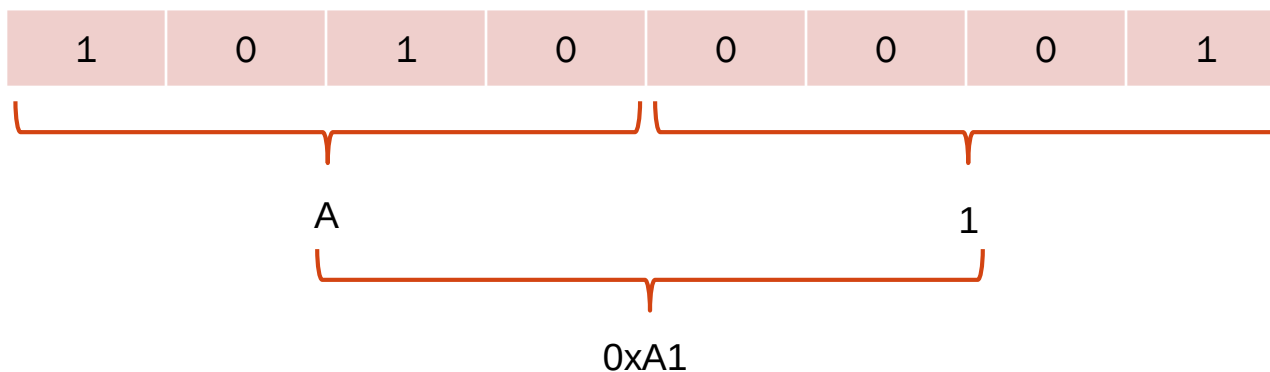
Resto: 1

Resto: A

Result: 0xA1

Da binario ad esadecimale:

$$161_{10} \rightarrow A1_{16}$$



Conversione tra sistemi

Da binario a decimale:

$$1100_2 \rightarrow 12_{10}$$

$$1100_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8 + 4 + 0 + 0 = 12$$

Da decimale ad esadecimale:

divisione intera

$$x // y \rightarrow x = y \cdot q + r$$

quoziente resto

digit position
// ← 1)

$$\begin{array}{rcl} 161 & : & 16 \\ 10 & : & 16 \end{array} = \begin{array}{rcl} 10 & & \\ 0 & & \end{array}$$

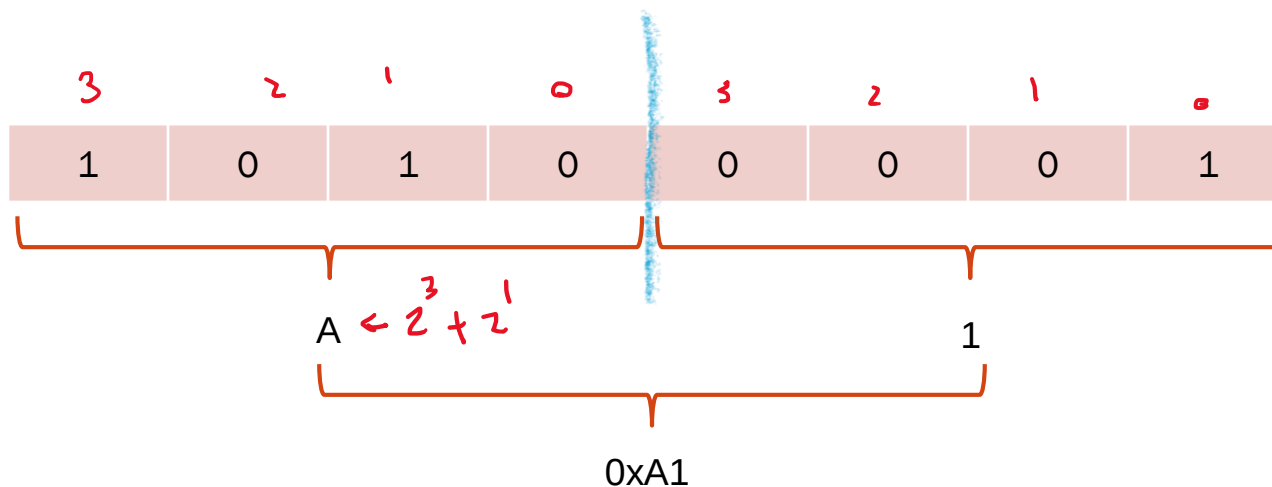
Resto: 1

Resto: A

Result: 0xA1

$$161_{10} \rightarrow A1_{16}$$

Da binario ad esadecimale:

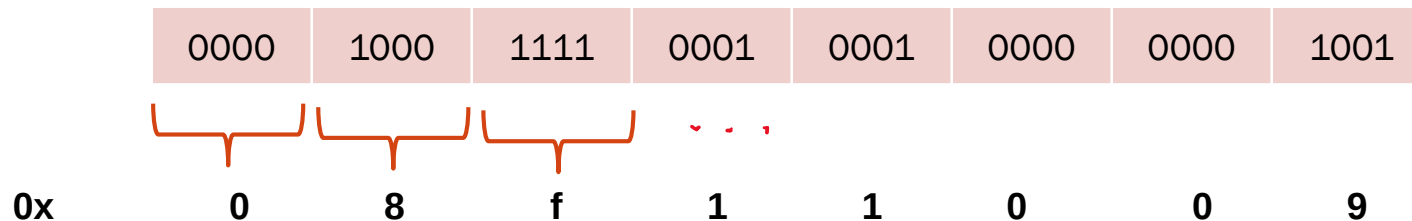


Perché usare il sistema 0x (esadecimale)?

Le istruzioni del calcolatore sono codificate in lunghe e «noiose» stringhe di numeri binari cioè

- Calcolatori hanno dimensioni dati multipla di 4 (ex. $32 \bmod 4 = 0$)
- I numeri hex (base $2^4=16$) sono convertibili in binario raggruppando cifre a **4**. Ossia un valore **hex** è possibile esprimerlo in **bin** su 4 bit.

*resto div.
intero*



Molto più facile leggere una rappresentazione compatta come 0x08f11009 che una lunga stringa di 0-1

Conversione tra sistemi

Massimo valore esprimibile con un byte:

$$11111111_2 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255 = 2^8 - 1$$

7 ← 8 bit → 0

Conversione di interi **senza segno**:

La parola (word) di default del RISC-V consta di 32 bit

Lunghezza	Valore min.	Valore max.
8 bit (1 byte)	0	$255 = 2^8 - 1$
16 bit (2 byte)	0	$65,535 = 2^{16} - 1$
32 bit (4 byte)	0	$4,294,967,295 = 2^{32} - 1$
64 bit (8 byte)	0	$18,446,744,073,709,551,615 = 2^{64} - 1$

Interi con segno

- Modulo e segno
 - $1000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000\ 0000_{\text{due}} = -256_{\text{dieci}}$
 - Intuitivo, ma dimezza il massimo valore esprimibile (inevitabile)
 - Ed ha due zeri!
- Complemento a due
 - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0000\ 0000_{\text{due}} = -256_{\text{dieci}}$
 - Meno intuitivo, ma molto pratico per somme e sottrazioni
- Complemento a uno
 - $1111\ 1111\ 1111\ 1111\ 1111\ 1110\ 0000\ 0000_{\text{due}} = -256_{\text{dieci}}$
 - Intuitivo, ma crea problemi con le somme
 - Ed ha due zeri!
- Notazione polarizzata
 - $0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0000\ 0000_{\text{due}} = -256_{\text{dieci}}$
 - Poco intuitivo, ma con il medesimo range del complemento a due
 - Lo zero è dato da $1000\dots000$
- Cosa preferirebbe un architetto di elaboratori? Perché?

Complemento a due nasce dall'hardware

La soluzione di default che l'hardware implementa naturalmente è il modo migliore per rappresentare i numeri interi negativi

0000	0000	0000	0000	0000	0000	0000	0000	_{due}	=	0 _{dec}
0000	0000	0000	0000	0000	0000	0000	0001	_{due}	=	1 _{dec}
0000	0000	0000	0000	0000	0000	0000	0010	_{due}	=	2 _{dec}
...									...	
0111	1111	1111	1111	1111	1111	1111	1101	_{due}	=	2 147 483 645 _{dec}
0111	1111	1111	1111	1111	1111	1111	1110	_{due}	=	2 147 483 646 _{dec}
0111	1111	1111	1111	1111	1111	1111	1111	_{due}	=	2 147 483 647 _{dec}
1000	0000	0000	0000	0000	0000	0000	0000	_{due}	=	-2 147 483 648 _{dec}
1000	0000	0000	0000	0000	0000	0000	0001	_{due}	=	-2 147 483 647 _{dec}
1000	0000	0000	0000	0000	0000	0000	0010	_{due}	=	-2 147 483 646 _{dec}
...									...	
1111	1111	1111	1111	1111	1111	1111	1101	_{due}	=	-3 _{dec}
1111	1111	1111	1111	1111	1111	1111	1110	_{due}	=	-2 _{dec}
1111	1111	1111	1111	1111	1111	1111	1111	_{due}	=	-1 _{dec}

Complemento a due nasce dall'hardware

La soluzione di default che l'hardware implementa naturalmente è il modo migliore per rappresenta i numeri interi negativi. Ad esempio in una word (32 bit) abbiamo:

$$(x_{31} \times -2^{31}) + (x_{30} \times 2^{30}) + (x_{29} \times 2^{29}) + \dots + (x_1 \times 2^1) + (x_0 \times 2^0)$$

ha il vantaggio che tutti i numeri negativi hanno il bit più significativo uguale a 1. Perciò l'hardware deve controllare soltanto questo bit per verificare se il numero è positivo o negativo (un numero che ha inizio con lo zero viene considerato positivo). Questo bit è a volte chiamato **bit di segno**.

Inoltre le operazioni di somma e sottrazione diventano banali (vedi dopo)

Complemento a due, esempio

La soluzione di default che l'hardware implementa naturalmente è il modo migliore per rappresenta i numeri interi negativi. Ad esempio in una word (32 bit) abbiamo:

1111 1111 1111 1111 1111 1111 1111 1100_{due}

Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$\begin{aligned} & (1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{31} + 2^{30} + 2^{29} + \dots + 2^2 + 0 + 0 \\ &= -2\,147\,483\,648_{\text{dec}} + 2\,147\,483\,644_{\text{dec}} \\ &= -4_{\text{dec}} \end{aligned}$$

Complemento a due, estensione del segno

Word a 32 bit in complemento a due quanto vale?

1111 1111 1111 1111 1111 1111 1111 1100_{due}

Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$\begin{aligned} & (1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{31} + 2^{30} + 2^{29} + \dots + 2^2 + 0 + 0 \\ &= -2\,147\,483\,648_{\text{dec}} + 2\,147\,483\,644_{\text{dec}} \\ &= -4_{\text{dec}} \end{aligned}$$

Cosa conterrà un registro a 32 bit 1111 1100_{due} se carichiamo dalla memoria il **byte** suddetto ?

Complemento a due, estensione del segno

Word a 32 bit in complemento a due quanto vale?

1111 1111 1111 1111 1111 1111 1111 1100_{due}

Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$\begin{aligned} & (1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{31} + 2^{30} + 2^{29} + \dots + 2^2 + 0 + 0 \\ &= -2\,147\,483\,648_{\text{dec}} + 2\,147\,483\,644_{\text{dec}} \\ &= -4_{\text{dec}} \end{aligned}$$

Cosa conterrà un registro a 32 bit 1111 1100_{due} se carichiamo dalla memoria il **byte** suddetto ?

→ Dipende da come lo carichiamo

Se lo interpretiamo, lo carichiamo senza segno con `load byte unsigned (lbu)`

Complemento a due, estensione del segno

Word a 32 bit in complemento a due quanto vale?

1111 1111 1111 1111 1111 1111 1111 1100_{due}

Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$\begin{aligned} & (1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{31} + 2^{30} + 2^{29} + \dots + 2^2 + 0 + 0 \\ &= -2\,147\,483\,648_{\text{dec}} + 2\,147\,483\,644_{\text{dec}} \\ &= -4_{\text{dec}} \end{aligned}$$

Cosa conterrà un registro a 32 bit 1111 1100_{due} se carichiamo dalla memoria il **byte** suddetto ?

→ Dipende da come lo carichiamo

Se lo interpretiamo, lo carichiamo senza segno con load byte unsigned (lbu)

1 _____ 0 _____ 1111 1100_{due}
31 9

Complemento a due, estensione del segno

Word a 32 bit in complemento a due quanto vale?

1111 1111 1111 1111 1111 1111 1111 1100_{due}

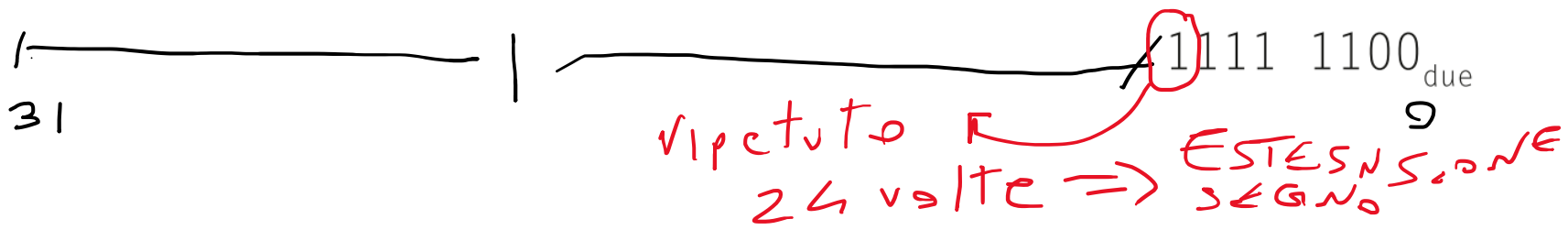
Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$\begin{aligned} & (1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \dots + (1 \times 2^1) + (0 \times 2^1) + (0 \times 2^0) \\ &= -2^{31} + 2^{30} + 2^{29} + \dots + 2^2 + 0 + 0 \\ &= -2\,147\,483\,648_{\text{dec}} + 2\,147\,483\,644_{\text{dec}} \\ &= -4_{\text{dec}} \end{aligned}$$

Cosa conterrà un registro a 32 bit 1111 1100_{due} se carichiamo dalla memoria il **byte** suddetto?

→ Dipende da come lo carichiamo

Se lo interpretiamo, lo carichiamo con segno con load byte (lb)



Complemento a due, scorciatoia

$$x + \bar{x} = -1, \quad \text{È la proprietà del complemento a due}$$

$$\text{ossia } \bar{x} + 1 = -x.$$

Scorciatoia per cambiare di segno un numero

Si cambi di segno il numero 2_{dec} e si verifichi poi il risultato cambiando di segno il numero -2_{dec} .

$$2_{\text{dec}} = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{\text{due}}$$

Invertendo questo numero tramite l'inversione dei bit e sommando poi 1, si ottiene:

$$\begin{array}{r} 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{\text{due}} \\ + 1_{\text{due}} \\ \hline = 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{\text{due}} \\ = -2_{\text{dec}} \end{array}$$

Complemento a due, somma e sottrazione

Basta eseguire la somma (ignorando l'overflow) e l'inversione di segno per rappresentare somme e sottrazioni tra numeri con segno!

0000	0000	0000	0000	0000	0000	0000	0000	_{due}	=	0 _{dec}
0000	0000	0000	0000	0000	0000	0000	0001	_{due}	=	1 _{dec}
0000	0000	0000	0000	0000	0000	0000	0010	_{due}	=	2 _{dec}
...									...	
0111	1111	1111	1111	1111	1111	1111	1101	_{due}	=	2 147 483 645 _{dec}
0111	1111	1111	1111	1111	1111	1111	1110	_{due}	=	2 147 483 646 _{dec}
0111	1111	1111	1111	1111	1111	1111	1111	_{due}	=	2 147 483 647 _{dec}
1000	0000	0000	0000	0000	0000	0000	0000	_{due}	=	-2 147 483 648 _{dec}
1000	0000	0000	0000	0000	0000	0000	0001	_{due}	=	-2 147 483 647 _{dec}
1000	0000	0000	0000	0000	0000	0000	0010	_{due}	=	-2 147 483 646 _{dec}
...									...	
1111	1111	1111	1111	1111	1111	1111	1101	_{due}	=	-3 _{dec}
1111	1111	1111	1111	1111	1111	1111	1110	_{due}	=	-2 _{dec}
1111	1111	1111	1111	1111	1111	1111	1111	_{due}	=	-1 _{dec}

Breve cenno ai numeri reali: standard IEEE754

Come rappresentiamo

3,14159265..._{dec} (π)

2,71828..._{dec} (e)

0,000000001_{dec} o $1,0_{dec} \times 10^{-9}$ (numero di secondi in un nanosecondo)

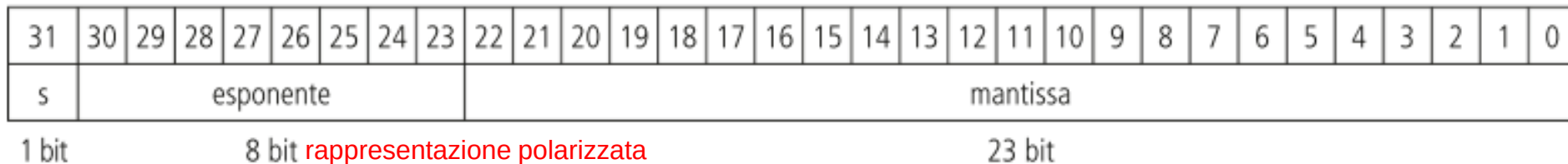
3 155 760 000_{dec} o $3,15576_{dec} \times 10^9$ (numero di secondi in un secolo medio)

Notazione Scientifica Normalizzata

$$(-1)^S \times F \times 2^E$$

Segno
Esponente
mantissa

Singola precisione, float (32bit)



- ↪

 - 2 bit speciali (infinito e denormalizzato)
 - i restanti 254 valori in notazione polarizzata, da -126 a +127

Come ripartiamo i 32 bit in esponente e mantissa? Compromesso fra **precisione del numero** (mantissa F) e **intervallo di valori** (esponente)

Breve cenno ai numeri reali: standard IEEE754

Come rappresentiamo

$3,14159265..._{\text{dec}} (\pi)$

$2,71828..._{\text{dec}} (e)$

$0,000000001_{\text{dec}}$ o $1,0_{\text{dec}} \times 10^{-9}$ (numero di secondi in un nanosecondo)

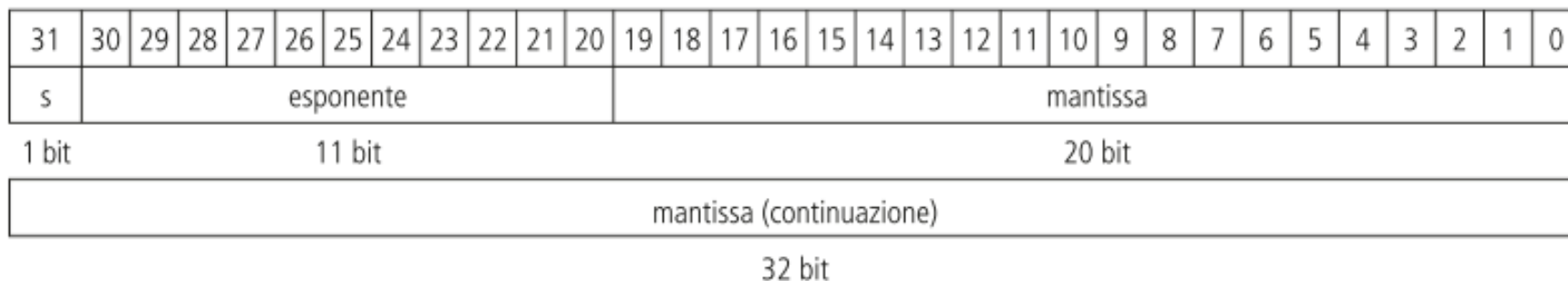
$3\,155\,760\,000_{\text{dec}}$ o $3,15576_{\text{dec}} \times 10^9$ (numero di secondi in un secolo medio)

Notazione Scientifica Normalizzata

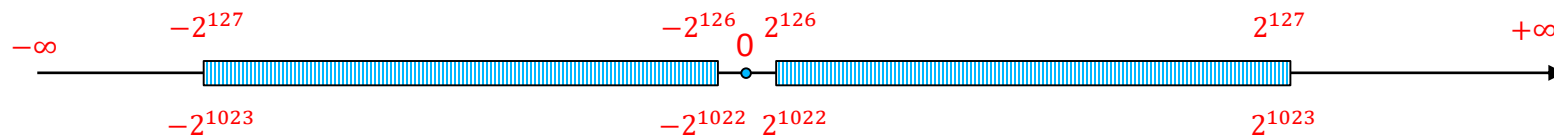
$$(-1)^S \times F \times 2^E$$

Segno
mantissa
Esponente

Doppia precisione, double (64bit)



Standard IEEE754



Singola precisione		Doppia precisione		Situazioni rappresentate
Esponente	Mantissa	Esponente	Mantissa	
0	0	0	0	0
0	Diverso da 0	0	Diverso da 0	$\pm$ numero denormalizzato
1–254	Qualsiasi numero	1–2046	Qualsiasi numero	$\pm$ numero in virgola mobile
255	0	2047	0	$\pm$ infinito
255	Diverso da 0	2047	Diverso da 0	NaN (<i>Not a Number</i>)

Figura 3.13 La codifica IEEE 754 dei numeri in virgola mobile. Un bit di segno, separato, determina il segno del numero. I numeri denormalizzati verranno descritti più avanti nell’approfondimento alla fine di questo paragrafo. Si possono trovare queste informazioni anche nella quarta colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro.

Il set di istruzioni RISC-V



SAPIENZA
UNIVERSITÀ DI ROMA

Architetture CISC e RISC

Architettura CISC

(Complex Instruction Set Computer)

- Istruzioni di **dimensione variabile**

Per il fetch della successiva è **necessaria la decodifica**

- **Formato variabile**

Decodifica complessa

- Operandi in memoria

Molti accessi alla memoria per istruzione

- **Pochi registri interni**

Maggior numero di accessi in memoria

- **Modi di indirizzamento complessi**

Maggior numero di accessi in memoria

Durata variabile della istruzione

Conflitti tra istruzioni più complicati

- Istruz. Complesse: pipeline più complicata

Architettura RISC

(Reduced Instruction Set Computer)

- Istruzioni di **dimensione fissa**

Fetch della successiva **senza decodifica della prec.**

- Istruzioni di **formato uniforme**

Per semplificare la fase di decodifica

- **Operazioni ALU solo tra registri**

Senza accesso a memoria

- **Molti registri interni**

Per i risultati parziali senza accessi alla memoria

- **Modi di indirizzamento semplici**

Con spiazzamento, 1 solo accesso a memoria

Durata fissa della istruzione

Conflitti semplici

- Istruz. semplici => pipeline più veloce

Istruz. complesse => molte più istruz. semplici ... MA: girano più velocemente !!!!

RISC-V – Ingredienti

Reduced Instruction Set Computer V

Operandi RISC-V

Nome	Esempio	Commenti
32 registri	x0-x31	Accesso veloce ai dati. Nel RISC-V gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni. Il registro x0 contiene sempre il valore 0
2 ³⁰ parole di memoria	Memoria[0], Memoria[4], ... Memoria[4 294 967 292]	Alla memoria si accede solamente attraverso istruzioni di trasferimento dati. Il RISC-V utilizza l'indirizzamento al byte, perciò due variabili successive) hanno indirizzi in memoria a distanza 4. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri

Importante:

- Memoria **RISC-V è indicizzata al byte**
- Se sono **all'indirizzo t** e devo leggere la parola successiva incremento indirizzo come **t+4**, questo perché una parola sono 4 byte ossia 32 bit (1 byte = 8 bit)
- **Il RISC-V non ha il vincolo di allineamento:** le parole (**DATI**) non devono iniziare ad indirizzi multipli di 4 (ma per motivi di prestazione e atomicità delle istruzioni si preferisce allineare)
- Gli indirizzi sono **little-endian** (byte meno significativo è memorizzato per primo, nella locazione di memoria con indirizzo minore)

RISC-V – Ingredienti: l'aritmetica è tra valori nei registri

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti

- Codice C

$f = (g + h) - (i + j);$

(assumendo che le variabili f, ..., j sono nei registri x19, x20, ..., x23)

- Codice RISC-V compilato

```
add x5, x20, x21
```

```
add x6, x22, x23
```

```
sub x19, x5, x6
```

RISC-V – Ingredienti: accesso alla memoria

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro
	Lettura parola, senza segno	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	sw x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una parola da registro a memoria
	Lettura mezza parola	lh x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	sh x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una mezza parola da registro a memoria
	Lettura byte	lb x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	sb x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola

RISC-V – Ingredienti

Linguaggio assembler RISC-V

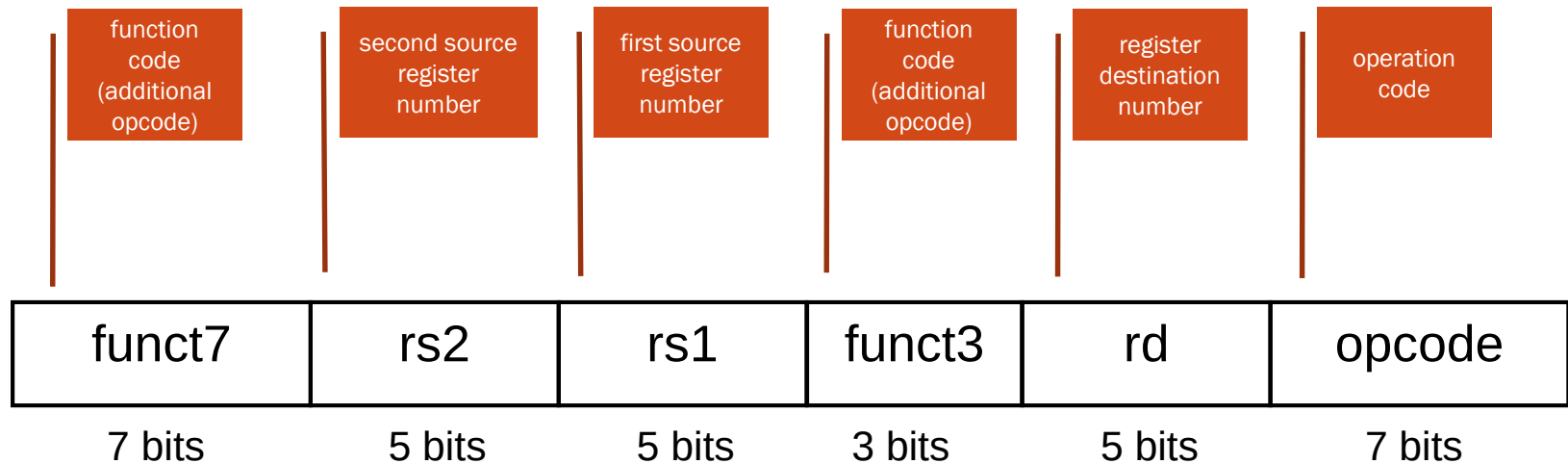
Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche	And	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit
	Or inclusivo	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit
	And immediato	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra un operando in registro e una costante
	Or esclusivo immediato	ori x5, x6, 20	$x5 = x6 20$	OR bit a bit tra un operando in registro e una costante
	Xor immediato	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	sra x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	srli x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	srai x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra aritmetico mediante costante

RISC-V – Ingredienti

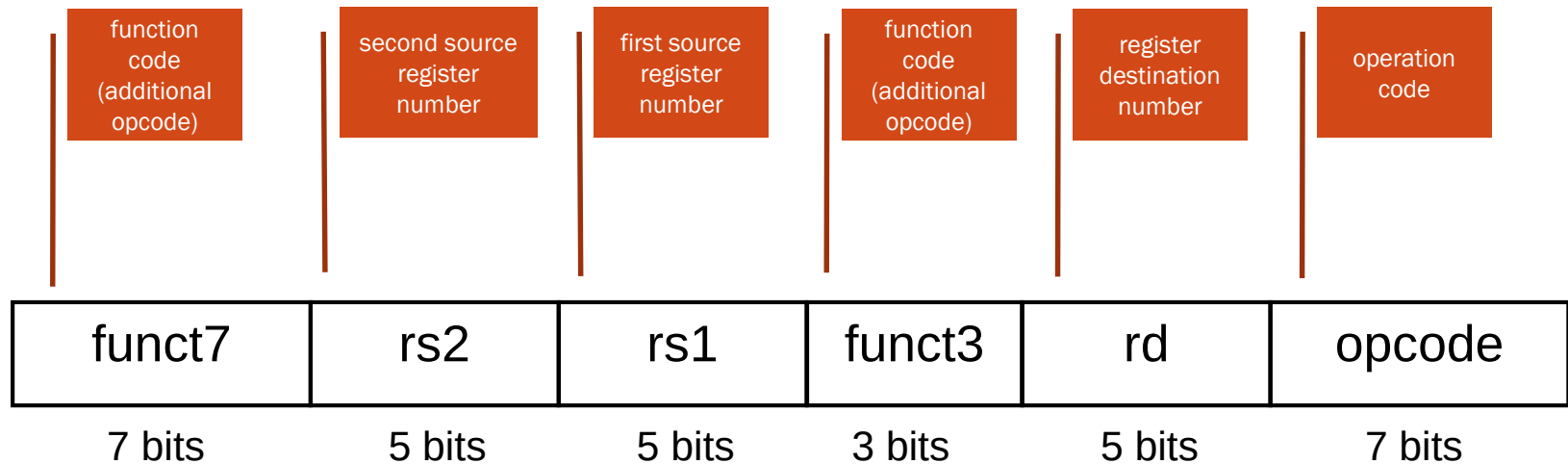
Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Salti condizionati	Salta se uguale	beq x5, x6, 100	Se (x5 == x6) vai a PC+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne x5, x6, 100	Se (x5 != x6) vai a PC+100	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	blt x5, x6, 100	Se (x5 < x6) vai a PC+100	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	bge x5, x6, 100	Se (x5 >= x6) vai a PC+100	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di, senza segno	bltu x5, x6, 100	Se (x5 < x6) vai a PC+100	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di, senza segno	bgeu x5, x6, 100	Se (x5 >= x6) vai a PC+100	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	jal x1, 100	x1 = PC+4; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	x1 = PC+4; vai a x5+100	Ritorno da procedura; chiamata indiretta

Rappresentazione dell'istruzione Registro (R-type)



Rappresentazione dell'istruzione Registro (R-type)



Esempio

`add x9, x20, x21`

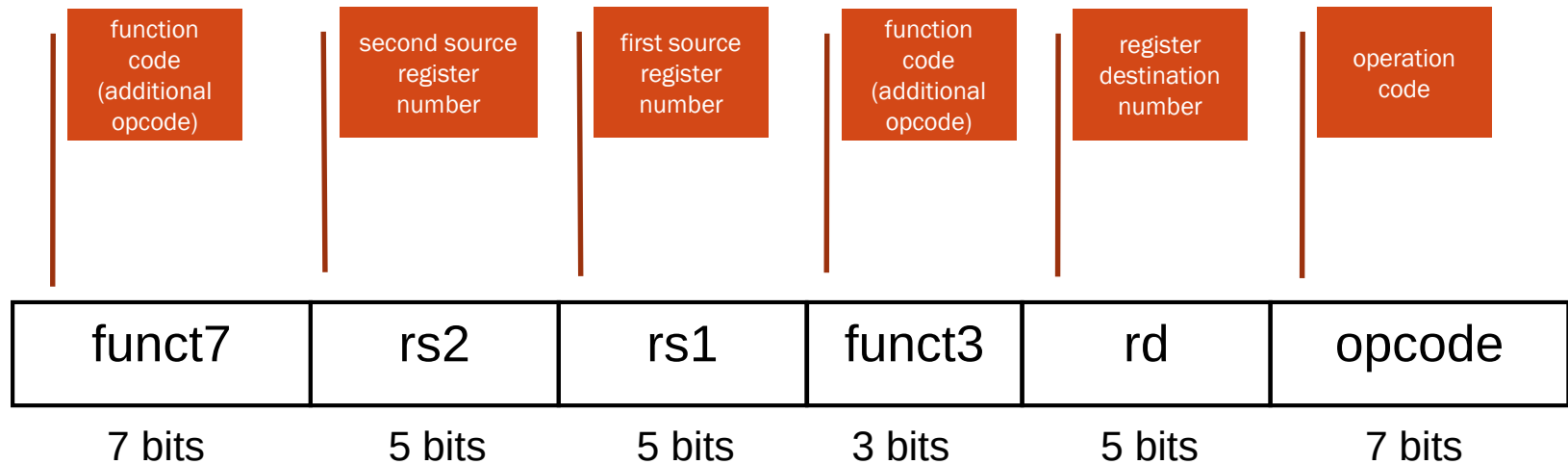
somma

opcode per operazioni aritmetiche

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011

0000 0001 0101 1010 0000 0100 1011 0011_{two} =
015A04B3₁₆

Rappresentazione dell'istruzione Registro (R-type)



Esempio 2

sub x9, x20, x21

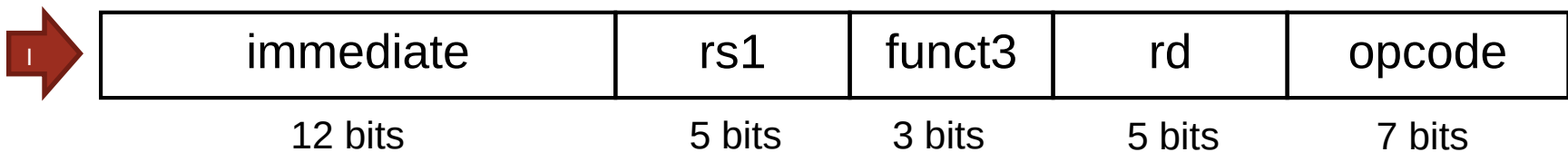
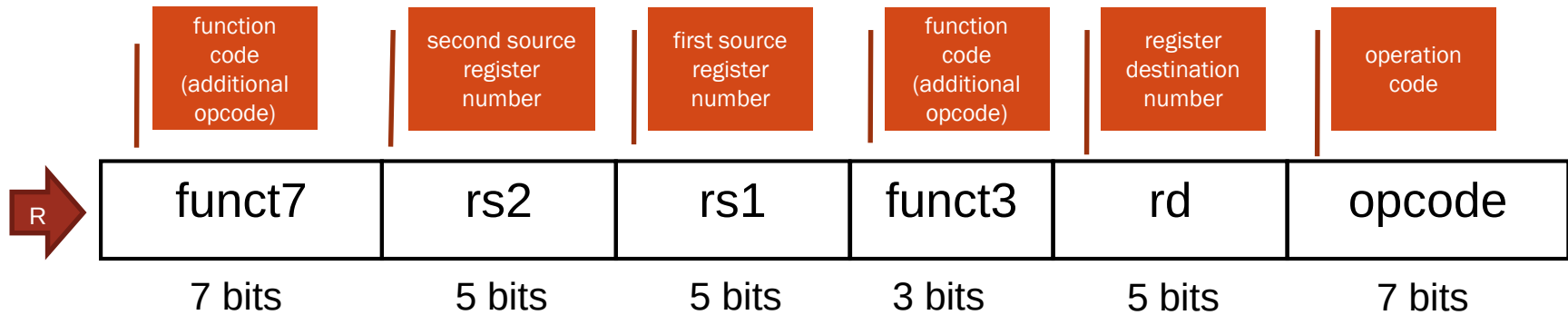
sottrazione

opcode per operazioni aritmetiche

32	21	20	0	9	51
0100000	10101	10100	000	01001	0110011

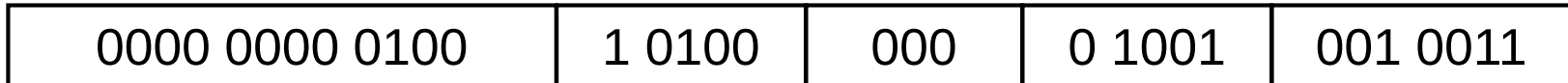
0100 0001 0101 1010 0000 0100 1011 0011_{two} =
415A04B3₁₆

Rappresentazione dell'istruzione Immediato (I-type)

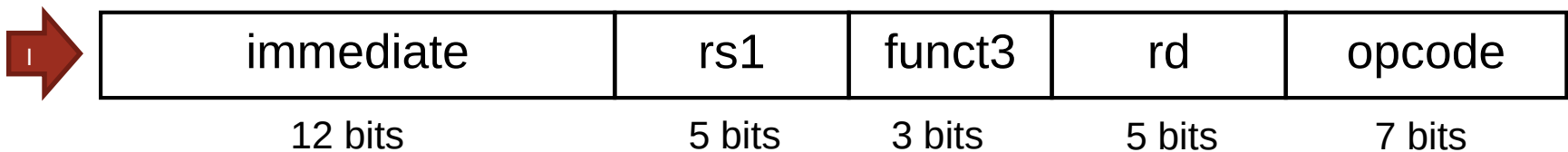
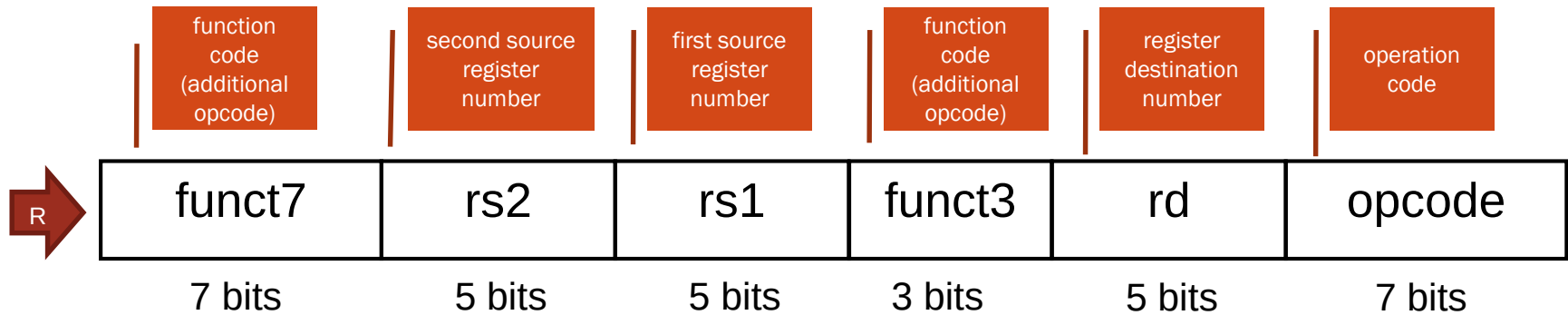


addi x9, x20, 4

0x...?

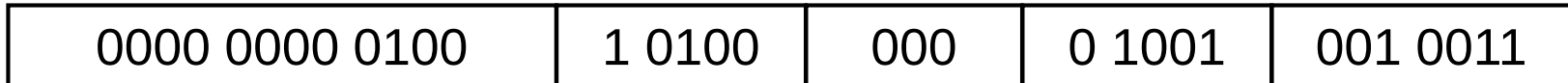


Rappresentazione dell'istruzione Immediato (I-type)

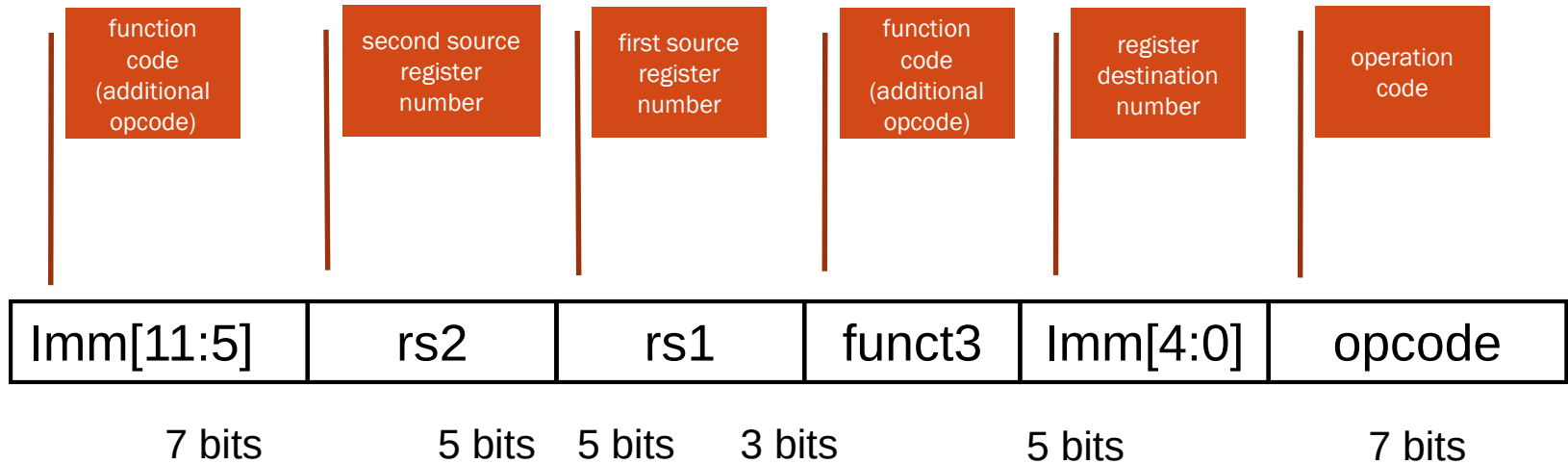
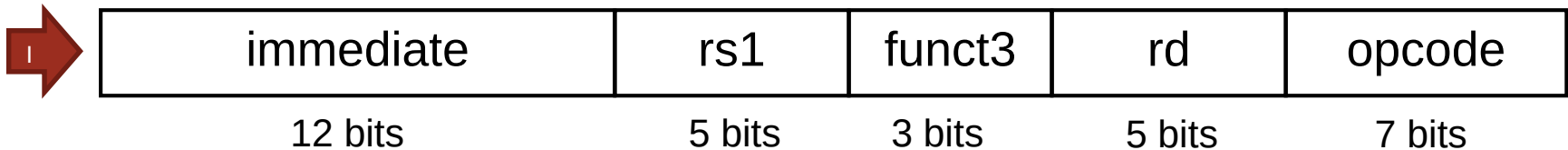
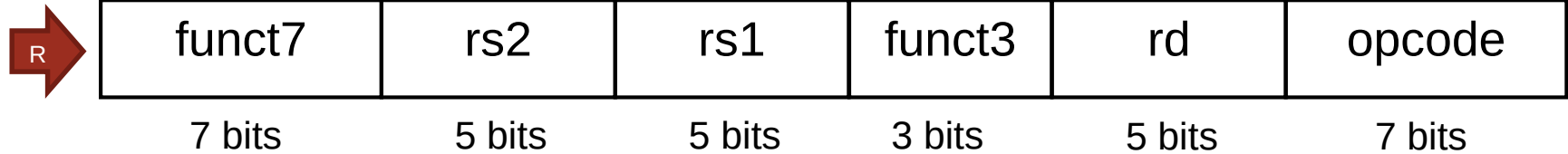


addi x9, x20, 4

0x004A0493



Rappresentazione dell'istruzione Store (S-type)



- I 12 bit di indirizzo sono divisi in due campi per preservare le posizioni di rs1 e rs2

Formato istruzioni R – I – S

Istruzione	Formato	funz7	rs2	r1	funz3	rd	codop
add (somma)	R	0000000	reg	reg	000	reg	0110011
sub (sottrazione)	R	0100000	reg	reg	000	reg	0110011
Istruzione	Formato	immediato		r1	funz3	rd	codop
addi (somma immediata)	I	costante		reg	000	reg	0010011
lw (load parola)	I	indirizzo		reg	010	reg	0000011
Istruzione	Formato	immediato	rs2	r1	funz3	immediato	codop
sw (store parola)	S	indirizzo	reg	reg	010	indirizzo	0100011

Figura 2.5 Codifica delle istruzioni RISC-V. In questa tabella “reg” indica il numero di un registro ed è compreso tra 0 e 31, e “indirizzo” è un indirizzo su 12 bit o una costante. I campi funz3 e funz7 agiscono come codici operativi aggiuntivi.

Formato delle istruzioni RISC-V

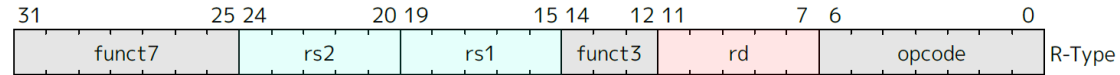
Istruzioni della CPU tutte da 32 bit con formato molto simile

R-type: (tipo a Registro)

- SENZA accesso alla memoria
- Istruzioni Aritmetico/Logiche

Esempio: **add x9, x20, x21**

$x9 \leftarrow x20 + x21$



I-type: (tipo Immediato)

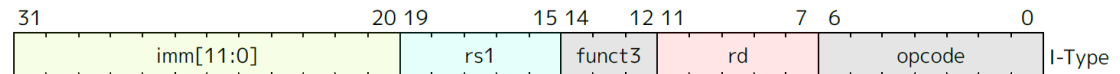
- Load
- Somma immediata

Esempi: **addi x1, x2, 1**

$x1 \leftarrow x2 + 1$

lw x9, 120(x10)

$x9 \leftarrow \text{MEM}[x10 + 120]$

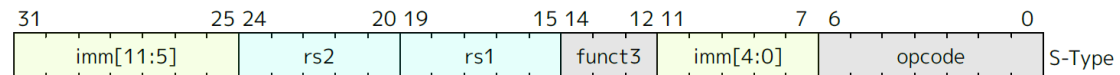


S-type: (tipo Store)

- Salvataggio registro in memoria

Esempio: **sw x9, 120(x10)**

$\text{MEM}[x10 + 120] \leftarrow x9$



Per la prossima lezione: RARS

Run speed at max (no interaction) Dec: Hex: Bin: ASCII:

Edit Execute

riscv1.s

```
1 lw t1, 0x10010001
```

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7ffffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x10010000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400008

Line: 1 Column: 17 ☒ Show Line Numbers

Messages Run I/O Program Arguments

☐ Popups
☒ Interactive
☐ Batch

Clear output
Clear input

Runtime exception at 0x00400004: address out of range 0x01001000
-- program is finished running (dropped off bottom) ---

Runtime exception at 0x00400004: Load address not aligned to word boundary 0x10010001

<https://github.com/rarsm/rars/releases/download/latest/rars-flatlaf.jar>
(testato con Java jdk23)